

Introdução

- Testes são essenciais para garantir a qualidade do código.
- No entanto:
 - Escrever testes de unidade pode se tornar um desafio devido ao código de preparação que deve ser escrito: inicialização de objetos, povoamento do banco de dados, comunicação remota, classes colaboradoras, ...
 - A medida que a quantidade de testes cresce, a sua execução pode levar um tempo considerável devido à comunicação com o banco de dados ou outros serviços externos.
- A principal razão dos testes de unidade serem difíceis de escrever e lentos de executar é por causa de suas classes colaboradoras.
 - Esse é um fato relevante em programas fortemente dependentes de sistemas de banco dados. É comum que as regras de negócio dependam de informações armazenadas.

Introdução

- A colaboração é geralmente um elemento bom e necessário, pois mantém a coesão da classe. No entanto, pode acarretar em alguns problemas:
 - Ao implementar a classe a ser testada, o desenvolvedor deve pensar e escrever os detalhes de todos os colaboradores envolvidos.
 - Criar e inicializar os colaboradores torna os testes de uma classe mais complicados pois alguns objetos requerem inicialização complexa até atingirem o estado correto para o teste.
 - Colaboradores introduzem acoplamento indesejado. Por exemplo, utilizar implementações reais de DAOs acoplaria o modelo de domínio ao banco de dados e nos forçaria a resolver questões de persistência.

Benefícios ao utilizar mock objects

- Testes de unidade devem ser isolados dos agentes externos ao código que está sendo testado.
 - O objetivo é testar o método em questão, e não os objetos externos.
- Mock objects permitem o isolamento dos testes de unidade fornecendo implementações falsas dos objetos colaboradores da unidade em teste.
- Quando utilizado com TDD, mock objects permitem que o desenvolvedor concentre-se na implementação do código em questão, livrando-o de escrever o código dos colaboradores.

Mockito

- Existem vários frameworks em Java para facilitar a geração de mock objects
 - **Mockito**, JMock, EasyMock, ...
- Vantagens do Mockito
 - Escrita clara.
 - API simples.
 - Permite a criação de mocks a partir de classes concretas.

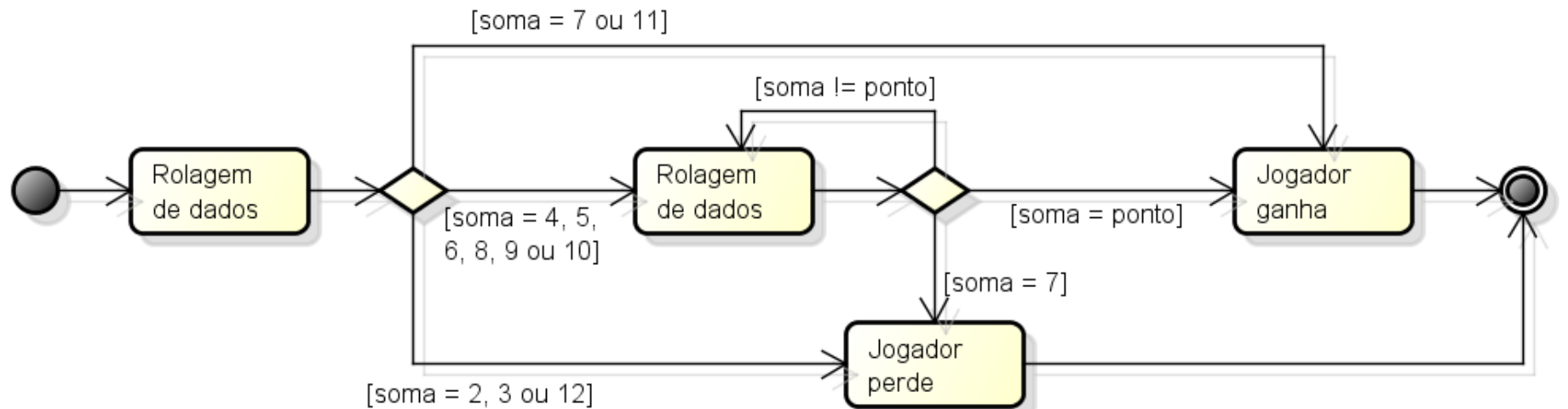


EXEMPLO 1 – JOGO CRAPS

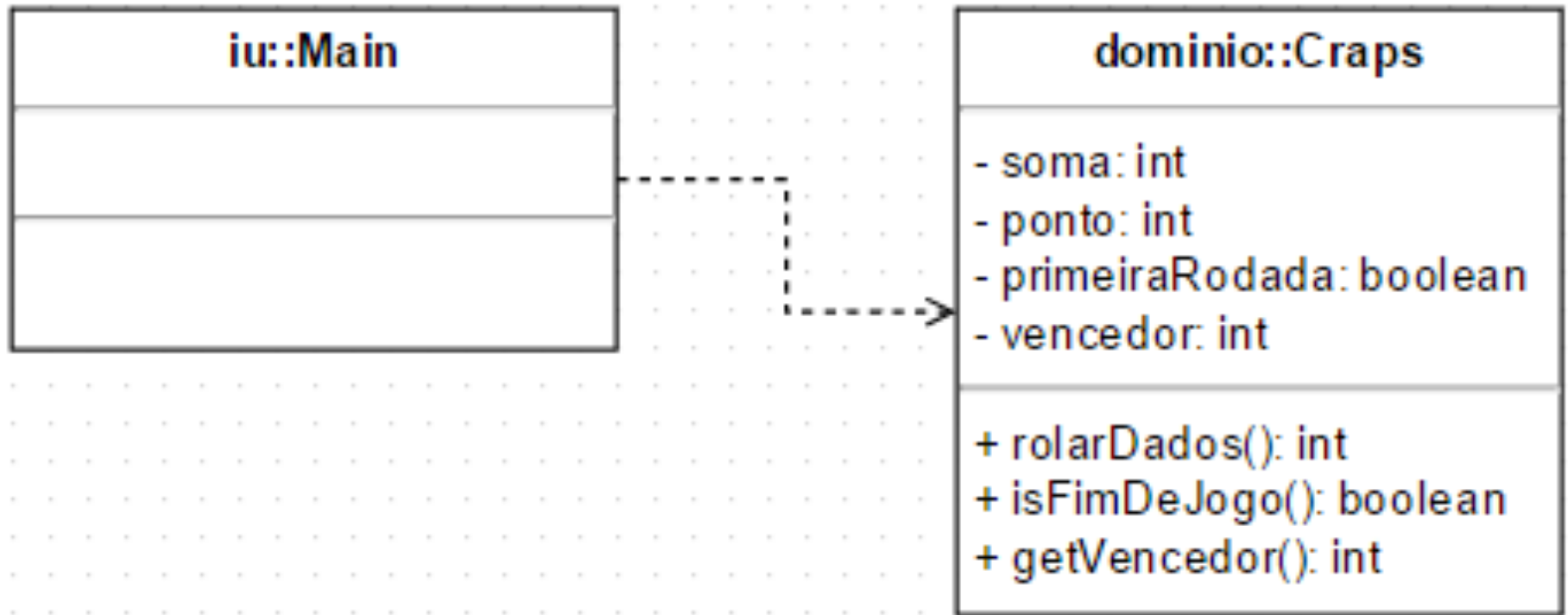
Introdução

- Este é um exemplo simples para demonstrar o uso do Mockito e de objetos mock.
- Craps (jogo de dados):
 - O jogador lança dois dados de seis faces. Depois que os dados param de rolar, calcula-se a soma dos pontos obtidos nos dois dados. Se a soma for 7 ou 11 no primeiro lance, o jogador ganha. Se a soma for 2, 3 ou 12 no primeiro lance, o jogador perde (a mesa ganha).
 - Se a soma for 4, 5, 6, 8, 9 ou 10 no primeiro lance, essa soma torna-se o ponto do jogador. Para ganhar, o jogador deve continuar a rolar os dados até atingir seu ponto (isto é, a soma deve ser igual ao ponto do jogador).
 - O jogador perde se obtiver um 7 antes de atingir seu ponto.

Fluxo do jogo



Projeto – visão estática



Fluxo da aplicação

```
public class Main{
    public static void main(String[] args){
        Craps craps = new Craps();
        Scanner scan = new Scanner(System.in);
        while(!craps.isFimDeJogo()){
            System.out.println("Digite enter para continuar.");
            scan.nextLine(); int soma = craps.rolarDados();
            System.out.println("Soma: " + soma);
        }
        scan.close();
        if(craps.getVencedor() == 1)
            System.out.println("Você ganhou!");
        else if(craps.getVencedor() == 2)
            System.out.println("A banca ganhou!");
        else
            throw new IllegalStateException();
        }
    }
```

Classe Craps (1)

```
import java.util.Random;
public class Craps{
    private int soma, ponto, vencedor;
    private boolean primeiraRodada = true;
    private Random rand = new Random();
    private int rolarUmDado(){
        return rand.nextInt(6) + 1;
    }
    public boolean isFimDeJogo(){
        return vencedor == 1 || vencedor == 2;
    }
    public int getVencedor(){
        return vencedor;
    }
}
```

Classe Craps (2)

```
public int rolarDados(){
    soma = rolarUmDado() + rolarUmDado();
    if(primeiraRodada){
        if(soma == 7 || soma == 11)
            vencedor = 1;
        else if(soma == 2 || soma == 3 || soma == 12)
            vencedor = 2;
        else
            ponto = soma;
            primeiraRodada = false;
    }
    else{
        if(soma == ponto)
            vencedor = 1;
        else if(soma == 7)
            vencedor = 2;
    }
    return soma;
}
```

Teste 1: jogador perde na primeira rolagem de dados

```
package craps.dominio;
import static org.junit.Assert.*;
import org.junit.Test;

public class CrapsTest {
    @Test
    public void testRolarDados_1(){
        Craps craps = new Craps();
        craps.rolarDados();
        assertTrue(craps.isFimDeJogo());
        assertEquals(2, craps.getVencedor());
    }
}
```

Problema com o teste 1

- Como garantir a obtenção de 2, 3 ou 12 na primeira rolagem já que a geração dos números é aleatória?
- Solução: substituir o objeto gerador de números aleatórios (**rand**) por um objeto mock.
 - Para tal, é necessário implementar o método **setRand** na classe **Craps**.

Teste 1 usando objeto mock

```
@Test
public void testRolarDados_1(){
    Random randMock = Mockito.mock(Random.class);
    //notar que são feitas duas rolagens de dado
    Mockito.when(randMock.nextInt(6)).thenReturn(0);
    Craps craps = new Craps();
    craps.setRand(randMock); craps.rolarDados();
    assertTrue(craps.isFimDeJogo());
    assertEquals(2, craps.getVencedor());
}
```

Teste 1 usando objeto mock

Criação do objeto mock.

```
@Test
public void testRolarDados_1(){
    Random randMock = Mockito.mock(Random.class);
    //notar que são feitas duas rolagens de dado
    Mockito.when(randMock.nextInt(6)).thenReturn(0);
    Craps craps = new Craps();
    craps.setRand(randMock); craps.rolarDados();
    assertTrue(craps.isFimDeJogo());
    assertEquals(2, craps.getVencedor());
}
```

Teste 1 usando objeto mock

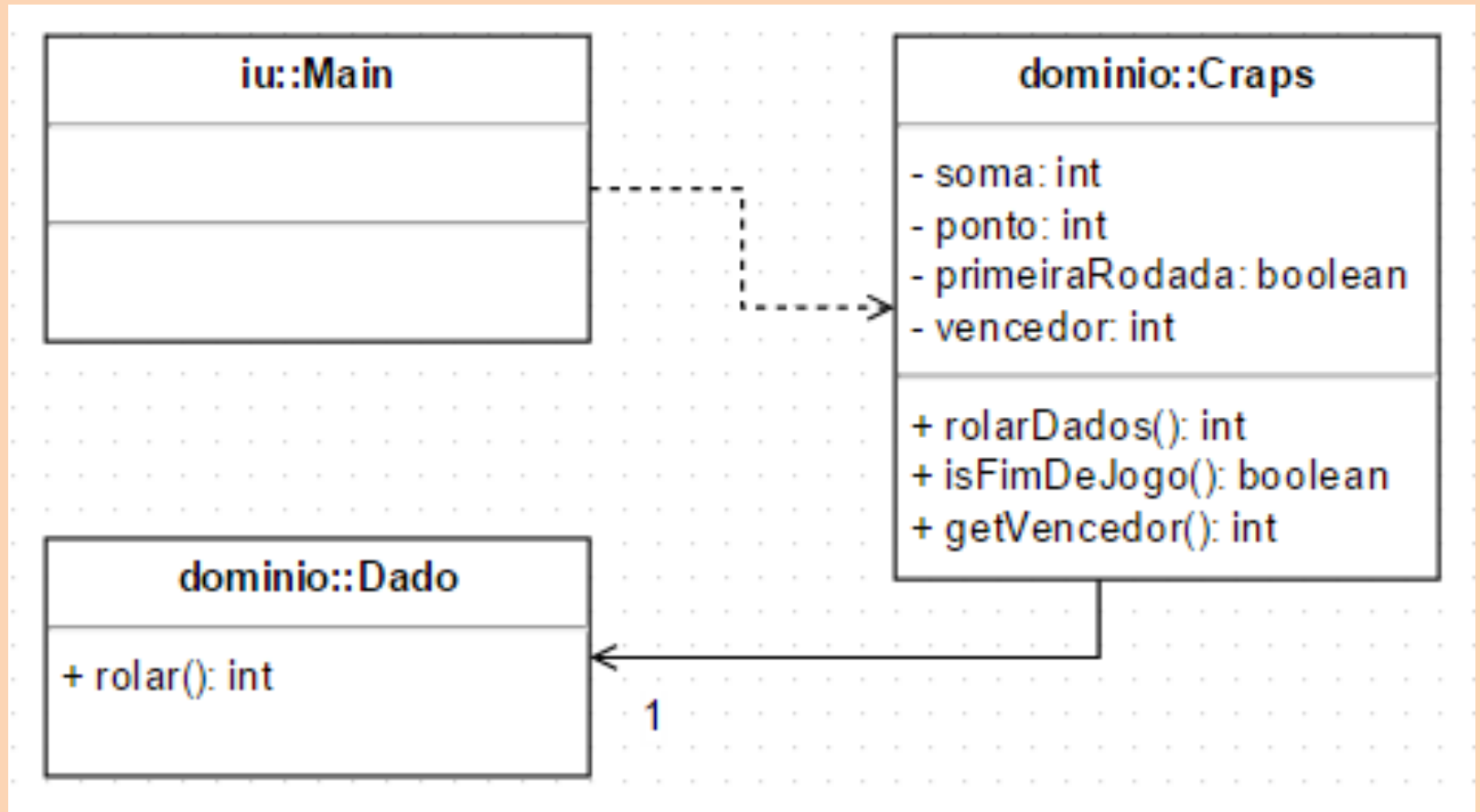
```
@Test
public void testRolarDados_1(){
    Random randMock = Mockito.mock(Random.class);
    //notar que são feitas duas rolagens de dado
    Mockito.when(randMock.nextInt(6)).thenReturn(0);
    Craps craps = new Craps();
    craps.setRand(randMock); craps.rolarDados();
    assertTrue(craps.isFimDeJogo());
    assertEquals(2, craps.getVencedor());
}
```

Configuração de comportamento do objeto mock. Neste caso, foi definido que as chamadas ao método **nextInt** retornem zero como resultado. Note que a configuração do mock foi realizada com base no código do método **rolarUmDado** da classe **Craps** para que a soma resultante seja igual a dois.

Melhorando o projeto

- A configuração de comportamento do mock foi baseada no código do método **rolarUmDado** da classe **Craps**. Além de ferir o encapsulamento, a configuração do mock não ficou intuitiva.
- Vamos melhorar o projeto movendo a geração dos números aleatórios para a classe **Dado**. Esta mudança melhora a coesão da classe **Craps** e melhora a legibilidade do teste.

Visão estática atualizada



Classe Dado

```
package craps.dominio;  
import java.util.Random;  
  
public class Dado {  
    private Random rand = new Random();  
    public int rolar(){  
        return rand.nextInt(6) + 1;  
    }  
}
```

Classe Craps atualizada (1)

```
public class Craps {  
    private int soma, ponto, vencedor;  
    private boolean primeiraRodada = true;  
    private Dado dado = new Dado();  
    public boolean isFimDeJogo(){  
        return vencedor == 1 || vencedor == 2;  
    }  
    public int getVencedor(){  
        return vencedor;  
    }  
    void setDado(Dado novoDado) {  
        this.dado = novoDado;  
    }  
}
```

Classe Craps atualizada (2)

```
public int rolarDados(){
    soma = dado.rolar() + dado.rolar();
    if(primeiraRodada){
        if(soma == 7 || soma == 11)
            vencedor = 1;
        else if(soma == 2 || soma == 3 || soma == 12)
            vencedor = 2;
        else
            ponto = soma;
            primeiraRodada = false;
    }
    else{
        if(soma == ponto)
            vencedor = 1;
        else if(soma == 7)
            vencedor = 2;
    }
    return soma;
}
```

Teste 1 atualizado

```
@Test
public void testRolarDados_1(){
    Dado dadoMock = Mockito.mock(Dado.class);
    //notar que são feitas duas rolagens de dado
    Mockito.when(dadoMock.rolar()).thenReturn(1);
    Craps craps = new Craps();
    craps.setDado(dadoMock);

    craps.rolarDados();

    assertTrue(craps.isFimDeJogo());
    assertEquals(2, craps.getVencedor());
}
```

Teste 2: jogador ganha na primeira rolagem

```
@Test
public void testRolarDados_2(){
    Dado dadoMock = Mockito.mock(Dado.class);
    //notar que são feitas duas rolagens de dado
    Mockito.when(dadoMock.rolar()).thenReturn(5, 2);
    Craps craps = new Craps();
    craps.setDado(dadoMock);

    craps.rolarDados();

    assertTrue(craps.isFimDeJogo());
    assertEquals(1, craps.getVencedor());
}
```

Teste 2: jogador ganha na primeira rolagem

```
@Test
public void testRolarDados_2(){
    Dado dadoMock = Mockito.mock(Dado.class);
    //notar que são feitas duas rolagens de dado
    Mockito.when(dadoMock.rolar()).thenReturn(5, 2);
    Craps craps = new Craps();
    craps.setDado(dadoMock);

    craps.rolarDados();

    assertTrue(craps.isFimDeJogo());
    assertEquals(1, craps.getVencedor());
}
```

Configuração do seguinte comportamento: retorne, respectivamente, 5 e 2 na primeira e segunda chamada ao método **rolar**.

Teste 3: jogador perde na terceira rolagem

```
@Test
public void testRolarDados_3(){
    Dado dadoMock = Mockito.mock(Dado.class);
    Craps craps = new Craps();
    craps.setDado(dadoMock);

    Mockito.when(dadoMock.rolar()).thenReturn(3, 5);
    craps.rolarDados();
    assertFalse(craps.isFimDeJogo());

    Mockito.when(dadoMock.rolar()).thenReturn(6, 5);
    craps.rolarDados();
    assertFalse(craps.isFimDeJogo());

    Mockito.when(dadoMock.rolar()).thenReturn(5, 2);
    craps.rolarDados();
    assertTrue(craps.isFimDeJogo());
    assertEquals(2, craps.getVencedor());
}
```

Teste 4: jogador ganha na segunda rolagem

```
@Test
public void testRolarDados_4(){
    Dado dadoMock = Mockito.mock(Dado.class);
    Craps craps = new Craps();
    craps.setDado(dadoMock);

    Mockito.when(dadoMock.rolar()).thenReturn(1, 3);
    craps.rolarDados();
    assertFalse(craps.isFimDeJogo());

    Mockito.when(dadoMock.rolar()).thenReturn(2, 2);
    craps.rolarDados();
    assertTrue(craps.isFimDeJogo());
    assertEquals(1, craps.getVencedor());
}
```